

STL in C++

1. Overview

The **STL**, or **Standard Template Library**, is a major part of C++ that provides ready-made data structures and algorithms.

Instead of implementing dynamic arrays, stacks, queues, maps, sets, heaps, and searching logic from scratch, C++ gives these through the STL.

In this guide, we focus on:

`vector`, `string`, `pair`, `tuple`, `stack`, `queue`, `deque`, `priority_queue`, `set`, `map`, `unordered_set`, `unordered_map`, iterators, and `lower_bound`.

Definition

The STL is a collection of:

Component	Meaning
Containers	Store data, like <code>vector</code> , <code>set</code> , <code>map</code>
Algorithms	Work on data, like <code>sort</code> , <code>lower_bound</code> , <code>reverse</code>
Iterators	Act like generalized pointers to traverse containers
Function objects	Callable objects used with STL algorithms

Why It Matters

STL matters because it lets you:

- Write shorter and cleaner code.
- Avoid bugs in common data structure implementations.
- Solve online assessment problems faster.
- Use optimized library implementations.
- Focus on logic instead of boilerplate.

Where It Is Used In Real Systems

STL-like data structures are used everywhere:

Real System	STL-Like Usage
Operating system	Queues for scheduling, maps for process tables
Database	Maps, sets, sorted indexes, hash tables
Browser	Stacks for history, maps for cache lookup
Backend server	Hash maps for sessions, queues for jobs

Real System	STL-Like Usage
Game engine	Vectors for entity lists, priority queues for pathfinding
Compiler	Symbol tables using maps or unordered maps

Why Interviewers Ask About It

Interviewers ask STL because it tests:

- Whether you know the right data structure for a problem.
- Whether you understand time complexity.
- Whether you can write fast code under pressure.
- Whether you know edge cases and iterator behavior.
- Whether you understand ordered vs unordered containers.

In placements, STL is not optional. It is the toolkit that lets you solve DSA problems efficiently.

2. Core Idea

The core idea of STL is:

Use the correct ready-made container or algorithm based on the operation you need most often.

Intuition

Every container has strengths and weaknesses.

For example:

- Need random access? Use **vector**.
- Need fast insertion/removal at both ends? Use **deque**.
- Need LIFO behavior? Use **stack**.
- Need FIFO behavior? Use **queue**.
- Need sorted unique elements? Use **set**.
- Need key-value mapping in sorted order? Use **map**.
- Need average $O(1)$ lookup? Use **unordered_map** or **unordered_set**.
- Need min/max element quickly? Use **priority_queue**.

Real-World Analogy

Think of STL containers as storage tools:

Tool	Analogy
vector	A row of numbered lockers
stack	A pile of plates
queue	A line at a ticket counter
deque	A train where coaches can be added at both ends

Tool	Analogy
<code>set</code>	A sorted attendance register with no duplicates
<code>map</code>	A dictionary sorted by word
<code>unordered_map</code>	A hash-based phone contact search
<code>priority_queue</code>	Hospital emergency queue

Small Example

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> marks = {70, 85, 90};
    marks.push_back(95);

    unordered_map<string, int> score;
    score["Amit"] = 85;
    score["Riya"] = 92;

    priority_queue<int> pq;
    pq.push(10);
    pq.push(30);
    pq.push(20);

    cout << marks[1] << "\n";           // 85
    cout << score["Riya"] << "\n";      // 92
    cout << pq.top() << "\n";           // 30
}
```

Step-By-Step Explanation

1. `vector<int>` stores integers dynamically.
2. `push_back` inserts at the end.
3. `unordered_map<string, int>` maps names to scores.
4. `score["Riya"]` gives value for key "Riya".
5. `priority_queue<int>` keeps the largest element at the top by default.
6. `pq.top()` returns the maximum element.

3. Important Subtopics

3.1 `vector`

`vector` is a dynamic array.

It stores elements contiguously in memory and supports random access.

```
vector<int> v;
v.push_back(10);
v.push_back(20);
cout << v[1]; // 20
```

Why it matters:

- Most commonly used STL container.
- Fast random access: $O(1)$.
- Fast insertion at end: amortized $O(1)$.
- Used heavily in arrays, graphs, DP, sorting, and binary search.

Common interview angle:

- Difference between `size()` and `capacity()`.
- Why `push_back` is amortized $O(1)$.
- Iterator invalidation after reallocation.

Important operations:

Operation	Complexity
<code>v[i]</code>	$O(1)$
<code>push_back</code>	Amortized $O(1)$
<code>pop_back</code>	$O(1)$
Insert in middle	$O(n)$
Delete from middle	$O(n)$

3.2 `string`

`string` is a class for handling character sequences.

```
string s = "placement";
cout << s.size(); // 9
cout << s.substr(0, 5); // place
```

Why it matters:

- Used in string matching, parsing, hashing, and text processing.
- Supports dynamic resizing like `vector<char>`.
- Common in coding rounds.

Common interview angle:

- `string` vs character array.

- `substr` complexity.
- Lexicographical comparison.

Important operations:

Operation	Meaning
<code>s.size()</code>	Number of characters
<code>s[i]</code>	Access character
<code>s.substr(pos, len)</code>	Extract substring
<code>s.find(x)</code>	Find substring or character
<code>s += t</code>	Append

3.3 `pair`

`pair` stores two values together.

```
pair<int, string> p = {1, "Amit"};
cout << p.first << " " << p.second;
```

Why it matters:

- Useful for coordinates, intervals, key-value results, and graph edges.
- Often used inside vectors, sets, maps, and priority queues.

Common interview angle:

- Sorting vector of pairs.
- Default comparison is lexicographical: first compares `first`, then `second`.

Example:

```
vector<pair<int, int>> intervals = {{2, 5}, {1, 3}, {1, 2}};
sort(intervals.begin(), intervals.end());
// Result: {1,2}, {1,3}, {2,5}
```

3.4 `tuple`

`tuple` stores more than two values together.

```
tuple<int, string, double> t = {1, "Amit", 8.5};
cout << get<1>(t); // Amit
```

Why it matters:

- Useful when each item has 3 or more fields.
- Common in graph edges: `{weight, node, parent}`.

Common interview angle:

- `pair` vs `tuple`.
- Sorting vector of tuples.
- Access using `get<index>()`.

3.5 `stack`

`stack` is a LIFO container adapter.

LIFO means **Last In, First Out**.

```
stack<int> st;
st.push(10);
st.push(20);
cout << st.top(); // 20
st.pop();
```

Why it matters:

- Used in recursion simulation, undo operations, expression evaluation, and monotonic stack problems.

Common interview angle:

- Balanced parentheses.
- Next greater element.
- Min stack.
- Infix, prefix, postfix expressions.

Important operations:

Operation	Complexity
<code>push</code>	$O(1)$
<code>pop</code>	$O(1)$
<code>top</code>	$O(1)$
<code>empty</code>	$O(1)$

3.6 `queue`

`queue` is a FIFO container adapter.

FIFO means **First In, First Out**.

```
queue<int> q;  
q.push(10);  
q.push(20);  
cout << q.front(); // 10  
q.pop();
```

Why it matters:

- Used in BFS, scheduling, buffering, and producer-consumer systems.

Common interview angle:

- BFS traversal.
- Level order traversal of binary tree.
- Rotten oranges problem.
- Sliding window processing.

3.7 deque

deque means **double-ended queue**.

It allows insertion and deletion at both front and back.

```
deque<int> dq;  
dq.push_back(10);  
dq.push_front(5);  
cout << dq.front(); // 5  
cout << dq.back(); // 10
```

Why it matters:

- More flexible than vector for front operations.
- Used in sliding window maximum.
- Underlying default container for **stack** and **queue**.

Common interview angle:

- **deque** vs **vector**.
- Sliding window maximum using monotonic deque.

3.8 priority_queue

priority_queue is a heap-based container adapter.

By default, it is a max heap.

```
priority_queue<int> pq;  
pq.push(30);
```

```

pq.push(10);
pq.push(20);
cout << pq.top(); // 30

```

Min heap:

```

priority_queue<int, vector<int>, greater<int>> minHeap;
minHeap.push(30);
minHeap.push(10);
cout << minHeap.top(); // 10

```

Why it matters:

- Used when you repeatedly need the largest or smallest element.
- Essential for Dijkstra, Prim, top K elements, and merging sorted lists.

Common interview angle:

- Max heap vs min heap.
- Custom comparator.
- Why `priority_queue` does not support random deletion.

Operations:

Operation	Complexity
<code>push</code>	$O(\log n)$
<code>pop</code>	$O(\log n)$
<code>top</code>	$O(1)$

3.9 set

`set` stores unique elements in sorted order.

Usually implemented as a balanced binary search tree.

```

set<int> s;
s.insert(30);
s.insert(10);
s.insert(10);

for (int x : s) cout << x << " "; // 10 30

```

Why it matters:

- Maintains sorted unique elements.
- Supports search, insert, delete in $O(\log n)$.

- Useful for problems needing sorted order and uniqueness.

Common interview angle:

- `set` vs `unordered_set`.
- `set` vs `multiset`.
- Using `lower_bound` with `set`.

3.10 `map`

`map` stores key-value pairs sorted by key.

Usually implemented as a balanced binary search tree.

```
map<string, int> age;
age["Amit"] = 21;
age["Riya"] = 22;
```

Why it matters:

- Maintains sorted keys.
- Useful when ordered traversal is needed.
- Search, insert, and delete are $O(\log n)$.

Common interview angle:

- `map` vs `unordered_map`.
- What happens when using `mp[key]` for a missing key.
- Frequency counting.

Important detail:

```
map<int, int> mp;
cout << mp[5]; // Inserts key 5 with default value 0
```

Use `find` if you only want to check existence.

3.11 `unordered_set`

`unordered_set` stores unique elements without sorted order.

It is usually implemented using hashing.

```
unordered_set<int> us;
us.insert(10);
us.insert(20);
cout << us.count(10); // 1
```

Why it matters:

- Average $O(1)$ lookup.
- Used for duplicate detection, visited checks, and membership tests.

Common interview angle:

- Average $O(1)$, worst-case $O(n)$.
- No sorted order.
- Hash collision.

3.12 unordered_map

`unordered_map` stores key-value pairs using hashing.

```
unordered_map<string, int> freq;
freq["apple"]++;
freq["banana"]++;
```

Why it matters:

- Most common container for frequency counting.
- Average $O(1)$ insertion and lookup.
- Used in two-sum, subarray sum, caching, and indexing.

Common interview angle:

- `unordered_map` vs `map`.
- Worst-case complexity.
- Custom hash for pairs.

3.13 Iterators

Iterators are objects that point to elements inside containers.

They act like generalized pointers.

```
vector<int> v = {10, 20, 30};

for (auto it = v.begin(); it != v.end(); it++) {
    cout << *it << " ";
}
```

Why it matters:

- STL algorithms use iterators.
- Needed for `sort`, `lower_bound`, `erase`, and traversal.
- Helps write generic code independent of container type.

Common interview angle:

- `begin()` points to first element.
- `end()` points one past the last element.
- Iterator invalidation.
- Difference between iterator and index.

3.14 `lower_bound`

`lower_bound` returns an iterator to the first element that is **greater than or equal to** a given value.

For vectors, the range must be sorted.

```
vector<int> v = {1, 3, 3, 5, 7};
auto it = lower_bound(v.begin(), v.end(), 3);
cout << (it - v.begin()); // 1
```

Why it matters:

- Used in binary search problems.
- Helps find positions efficiently.
- Useful for LIS, scheduling, allocation, and range queries.

Common interview angle:

- Difference between `lower_bound` and `upper_bound`.
- Works in $O(\log n)$ on random-access iterators like vector.
- For `set` and `map`, prefer member function: `s.lower_bound(x)`.

4. Real-World Example

Backend Job Scheduler

Imagine a backend server that processes user tasks:

- Each task has an ID.
- Tasks have priority.
- Completed users should not be processed again.
- User metadata must be retrieved quickly.
- Tasks may be processed in arrival order for normal queues.

STL usage:

Requirement	STL Container
Store all task IDs	<code>vector<int></code>
Process urgent tasks first	<code>priority_queue</code>
Process normal tasks in arrival order	<code>queue</code>

Requirement	STL Container
Avoid duplicate users	<code>unordered_set</code>
Store user ID to user info	<code>unordered_map</code>
Keep sorted active user IDs	<code>set</code>
Store timestamp to task	<code>map</code>

Example:

```

struct Task {
    int priority;
    int id;
};

struct CompareTask {
    bool operator()(const Task& a, const Task& b) {
        return a.priority < b.priority; // higher priority first
    }
};

priority_queue<Task, vector<Task>, CompareTask> urgentTasks;
unordered_set<int> processedUsers;
unordered_map<int, string> userName;

```

This resembles real job queues, notification systems, and backend worker services.

5. Diagrams / Mental Models

Choosing The Right Container

```

Need storage?
|
+-- Need index access?
|   |
|   +-- Yes -> vector
|
+-- Need add/remove from both ends?
|   |
|   +-- Yes -> deque
|
+-- Need LIFO?
|   |
|   +-- Yes -> stack
|
+-- Need FIFO?
|   |
|   +-- Yes -> queue
|

```

```

+-- Need highest/lowest priority repeatedly?
|   |
|   +-- Yes -> priority_queue
|
+-- Need uniqueness?
|   |
|   +-- Sorted -> set
|   +-- Fast average lookup -> unordered_set
|
+-- Need key-value?
|   |
|   +-- Sorted keys -> map
|   +-- Fast average lookup -> unordered_map

```

Ordered vs Unordered Containers

```

set/map
  Sorted order
  O(log n)
  Tree-based

unordered_set/unordered_map
  No sorted order
  Average O(1)
  Hash-table-based

```

Iterator Model

```

vector: [10] [20] [30] [40]
        ^
        begin()

vector: [10] [20] [30] [40]   one-past-end
                               ^
                               end()

```

Important: `end()` is not a valid element. Do not dereference it.

`lower_bound` Mental Model

```

v = [1, 3, 3, 5, 7]

lower_bound(v, 3) -> first position where value >= 3

Index:  0  1  2  3  4
Value:  1  3  3  5  7

```

^
answer

6. Common Interview Questions

Q1. What is STL in C++?

Answer:

STL is the Standard Template Library in C++. It provides ready-made generic containers, algorithms, and iterators. Examples include `vector`, `set`, `map`, `queue`, `sort`, and `lower_bound`.

Key points interviewer expects:

- STL is generic and template-based.
- It includes containers, algorithms, and iterators.
- It improves speed and reliability in coding.

Common mistakes:

- Saying STL is only data structures.
- Forgetting algorithms and iterators.

Q2. What is the difference between `vector` and array?

Answer:

An array has fixed size, while `vector` is dynamic and can resize automatically.

Feature	Array	Vector
Size	Fixed	Dynamic
Memory	Contiguous	Contiguous
Has methods	No	Yes
Resize	Manual or impossible	Automatic

Key points interviewer expects:

- Both support $O(1)$ random access.
- `vector` may reallocate memory when capacity grows.

Common mistakes:

- Saying vector is always faster than array.
- Ignoring reallocation cost.

Q3. Why is `vector::push_back` amortized $O(1)$?

Answer:

Most `push_back` operations insert directly at the end in $O(1)$. Sometimes, when capacity is full, vector allocates a larger memory block and copies or moves old elements. Since this expensive operation happens occasionally, the average cost over many insertions is $O(1)$.

Key points interviewer expects:

- Capacity grows geometrically.
- Occasional reallocation.
- Average over many operations is $O(1)$.

Common mistakes:

- Saying every `push_back` is strictly $O(1)$.
- Not mentioning capacity.

Q4. What is the difference between `set` and `unordered_set`?

Answer:

`set` stores unique elements in sorted order and usually uses a balanced BST. `unordered_set` stores unique elements without order and uses hashing.

Feature	<code>set</code>	<code>unordered_set</code>
Order	Sorted	Not sorted
Insert/search/delete	$O(\log n)$	Average $O(1)$
Implementation	Balanced BST	Hash table
Supports <code>lower_bound</code>	Yes	No meaningful sorted lower bound

Key points interviewer expects:

- Ordered vs unordered.
- $O(\log n)$ vs average $O(1)$.
- Worst-case hash table operations can be $O(n)$.

Common mistakes:

- Saying `unordered_set` is always $O(1)$.
- Expecting sorted traversal from `unordered_set`.

Q5. What is the difference between `map` and `unordered_map`?

Answer:

`map` stores key-value pairs sorted by key. `unordered_map` stores key-value pairs using hashing and does not maintain sorted order.

Key points interviewer expects:

- `map`: $O(\log n)$, sorted keys.
- `unordered_map`: average $O(1)$, no order.

- Choose based on whether ordering is needed.

Common mistakes:

- Using `unordered_map` when sorted output is required.
- Using `map` unnecessarily when only frequency counting is needed.

Q6. What happens when you access a missing key using `mp[key]`?

Answer:

In `map` and `unordered_map`, `mp[key]` inserts the key with a default value if the key does not already exist.

```
map<int, int> mp;  
cout << mp[10]; // inserts 10 with value 0
```

Key points interviewer expects:

- `operator[]` may modify the map.
- Use `find` or `count` for checking existence.

Common mistakes:

- Assuming `mp[key]` only checks existence.

Q7. What is the difference between `lower_bound` and `upper_bound`?

Answer:

`lower_bound(x)` returns the first element greater than or equal to `x`.

`upper_bound(x)` returns the first element strictly greater than `x`.

```
vector<int> v = {1, 3, 3, 5};  
  
lower_bound(v.begin(), v.end(), 3); // index 1  
upper_bound(v.begin(), v.end(), 3); // index 3
```

Key points interviewer expects:

- Range must be sorted for vector usage.
- Return value is an iterator.

Common mistakes:

- Confusing greater-than-or-equal with strictly greater.
- Dereferencing returned iterator without checking `end()`.

Q8. What is an iterator?

Answer:

An iterator is an object that points to an element in a container. It is used to traverse containers and connect containers with STL algorithms.

```
for (auto it = v.begin(); it != v.end(); it++) {  
    cout << *it;  
}
```

Key points interviewer expects:

- Works like a generalized pointer.
- `begin()` points to first element.
- `end()` points one past the last element.

Common mistakes:

- Dereferencing `end()`.
- Treating all iterators like vector iterators.

Q9. What is the difference between `stack` and `queue`?

Answer:

`stack` follows LIFO: last inserted element is removed first.

`queue` follows FIFO: first inserted element is removed first.

Feature	Stack	Queue
Principle	LIFO	FIFO
Insert	<code>push</code>	<code>push</code>
Access	<code>top</code>	<code>front</code> , <code>back</code>
Remove	<code>pop</code>	<code>pop</code>
Example	Undo, recursion	BFS, scheduling

Common mistakes:

- Trying to iterate directly over `stack` or `queue`.
- Forgetting `pop()` does not return the removed element.

Q10. How do you create a min heap in C++ STL?

Answer:

Use `priority_queue` with `greater<int>`.

```
priority_queue<int, vector<int>, greater<int>> minHeap;
```

Key points interviewer expects:

- Default `priority_queue` is max heap.
- Third template argument controls comparison.

Common mistakes:

- Writing only `priority_queue<int, greater<int>>`.
- Thinking `top()` removes the element.

Q11. How do you sort a vector of pairs?

Answer:

By default, pairs sort lexicographically: first by `first`, and if equal, by `second`.

```
vector<pair<int, int>> v = {{2, 3}, {1, 5}, {1, 2}};
sort(v.begin(), v.end());
// {1,2}, {1,5}, {2,3}
```

Key points interviewer expects:

- Default comparison exists.
- Custom comparator can change ordering.

Common mistakes:

- Assuming it sorts only by `first`.

Q12. Why should we prefer `s.lower_bound(x)` for a set instead of `lower_bound(s.begin(), s.end(), x)`?

Answer:

For `set`, the member function `s.lower_bound(x)` uses the tree structure and runs in $O(\log n)$. The generic algorithm `lower_bound(s.begin(), s.end(), x)` works with iterators and may take $O(n)$ iterator movement because set iterators are not random-access.

Key points interviewer expects:

- Member function is optimized for associative containers.
- Iterator category matters.

Common mistakes:

- Assuming all `lower_bound` calls are $O(\log n)$.

7. Deep-Dive Questions

Q1. What is iterator invalidation?

Iterator invalidation means an iterator becomes unusable because the container changed internally.

Example:

```
vector<int> v = {1, 2, 3};
auto it = v.begin();
v.push_back(4); // may reallocate
// it may now be invalid
```

For `vector`, reallocation can invalidate all iterators, pointers, and references.

For `set` and `map`, inserting usually does not invalidate existing iterators, but erasing an element invalidates iterator to that erased element.

Q2. Why is `unordered_map` worst-case $O(n)$?

`unordered_map` uses hashing. Ideally, keys distribute evenly across buckets, giving average $O(1)$ operations.

Worst case happens when many keys collide into the same bucket. Then lookup may scan many elements, making operations $O(n)$.

Interview point:

Average $O(1)$ is practical, but not guaranteed in the worst case.

Q3. How does a custom comparator work in `priority_queue`?

In `priority_queue`, the comparator decides which element has lower priority.

For pairs:

```
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>>
pq;
```

This creates a min heap by lexicographical pair order.

For custom type:

```
struct Compare {
    bool operator()(const pair<int, int>& a, const pair<int, int>& b) {
        return a.second > b.second; // smaller second comes first
    }
};
```

Q4. Why does `deque` allow efficient front insertion but `vector` does not?

`vector` stores elements in one contiguous block. Inserting at the front requires shifting all existing elements, so it is $O(n)$.

`deque` is usually implemented as multiple fixed-size blocks. It can add blocks at the front or back efficiently, so `push_front` and `push_back` are $O(1)$ amortized.

Q5. Can `lower_bound` be used on descending sorted data?

The default `lower_bound` assumes ascending order with the default comparator.

For descending order, provide a matching comparator:

```
vector<int> v = {9, 7, 5, 3};
auto it = lower_bound(v.begin(), v.end(), 6, greater<int>());
```

The comparator used in `lower_bound` must match the sorting order.

8. Comparison Tables

`vector` vs `deque`

Feature	<code>vector</code>	<code>deque</code>
Memory	Contiguous	Multiple blocks
Random access	$O(1)$	$O(1)$
Push back	Amortized $O(1)$	$O(1)$
Push front	$O(n)$	$O(1)$
Cache friendliness	Better	Usually weaker
Best use	Dynamic array	Front and back operations

`stack` vs `queue`

Feature	<code>stack</code>	<code>queue</code>
Rule	LIFO	FIFO
Access element	<code>top()</code>	<code>front()</code> , <code>back()</code>
Common use	Parentheses, recursion, undo	BFS, scheduling, buffering
Direct iteration	No	No

`set` vs `unordered_set`

Feature	set	unordered_set
Ordering	Sorted	No order
Duplicate values	Not allowed	Not allowed
Search	$O(\log n)$	Average $O(1)$
Lower bound	Supported meaningfully	Not supported by order
Implementation	Balanced BST	Hash table

map vs unordered_map

Feature	map	unordered_map
Key order	Sorted	No order
Search	$O(\log n)$	Average $O(1)$
Range queries	Good	Not suitable
Frequency counting	Works	Usually better
Worst case	$O(\log n)$	$O(n)$

pair vs tuple

Feature	pair	tuple
Number of values	Exactly 2	2 or more
Access	.first, .second	get<index>()
Readability	Better for two values	Can become less readable
Common use	Coordinates, intervals	Multi-field records

lower_bound vs upper_bound

Function	Returns
lower_bound(x)	First element $\geq x$
upper_bound(x)	First element $> x$
equal_range(x)	Range of elements equal to x

Example:

```
v = [1, 2, 2, 2, 5]

lower_bound(2) -> index 1
```

```
upper_bound(2) -> index 4
count of 2 = upper_bound - lower_bound = 3
```

9. Common Mistakes

- Using `unordered_map` when sorted output is required.
- Forgetting that `priority_queue` is max heap by default.
- Dereferencing `end()` iterator.
- Calling `pop()` and expecting it to return the removed element.
- Using `mp[key]` only to check if a key exists, accidentally inserting it.
- Assuming `unordered_map` is always $O(1)$.
- Using `lower_bound` on an unsorted vector.
- Forgetting `lower_bound` returns an iterator, not an index.
- Using generic `lower_bound` on `set` instead of `set.lower_bound`.
- Erasing from a vector while iterating incorrectly.
- Assuming `set` allows duplicates.
- Forgetting to include custom comparator for min heap.
- Using `tuple` where a named struct would be more readable.

10. Edge Cases / Special Cases

Empty Containers

Do not call `top`, `front`, `back`, or `pop` on an empty container.

```
if (!st.empty()) st.pop();
```

`lower_bound` May Return `end()`

```
auto it = lower_bound(v.begin(), v.end(), x);
if (it != v.end()) {
    cout << *it;
}
```

`vector` Reallocation

After `push_back`, old iterators may become invalid if capacity changes.

Use `reserve` if you know expected size:

```
vector<int> v;
v.reserve(100000);
```

map[key] Inserts Missing Key

Use:

```
if (mp.find(key) != mp.end()) {  
    // key exists  
}
```

Hashing Pairs

`unordered_map<pair<int, int>, int>` does not work directly without a custom hash in many standard setups.

Common workaround in contests:

```
map<pair<int, int>, int> mp;
```

or write a custom hash.

Duplicate Values

`set` removes duplicates.

If duplicates are required in sorted order, use `multiset`.

priority_queue Cannot Update Priority Directly

For algorithms like Dijkstra, commonly push a new pair and ignore stale entries when popped.

String Indexing

Accessing `s[i]` when `i >= s.size()` is invalid.

Iterator Erase Pattern

For vector:

```
v.erase(v.begin() + i);
```

For map/set while iterating:

```
for (auto it = s.begin(); it != s.end(); ) {  
    if (*it % 2 == 0) it = s.erase(it);  
    else it++;  
}
```

11. How to Explain in Interview

STL in C++ is a library of ready-made generic containers, algorithms, and iterators. I use containers like `vector` for dynamic arrays, `set` and `map` when sorted order matters, `unordered_map` and `unordered_set` for average $O(1)$ lookup, `stack` and `queue` for LIFO and FIFO behavior, and `priority_queue` for heap-based priority access. The main interview skill is choosing the right container based on time complexity, ordering requirement, duplicate handling, and access pattern.

12. Quick Revision Notes

Key Definitions

Term	Meaning
<code>vector</code>	Dynamic array
<code>string</code>	Dynamic character sequence
<code>pair</code>	Stores two values
<code>tuple</code>	Stores multiple values
<code>stack</code>	LIFO adapter
<code>queue</code>	FIFO adapter
<code>deque</code>	Double-ended queue
<code>priority_queue</code>	Heap-based priority structure
<code>set</code>	Sorted unique elements
<code>map</code>	Sorted key-value pairs
<code>unordered_set</code>	Hash-based unique elements
<code>unordered_map</code>	Hash-based key-value pairs
iterator	Generalized pointer
<code>lower_bound</code>	First element greater than or equal to target

Important Points

- `vector` gives $O(1)$ random access.
- `set` and `map` are sorted and $O(\log n)$.
- `unordered_set` and `unordered_map` are average $O(1)$, but unordered.
- `priority_queue` is max heap by default.
- `lower_bound` requires sorted data for vectors.
- `end()` is one past the last element.
- `pop()` removes but does not return.
- `mp[key]` inserts if key is missing.

Common Comparisons

Compare	Main Difference
<code>vector</code> vs <code>deque</code>	Contiguous dynamic array vs efficient both-end operations
<code>set</code> vs <code>unordered_set</code>	Sorted $O(\log n)$ vs average $O(1)$ unordered
<code>map</code> vs <code>unordered_map</code>	Sorted keys vs hash lookup
<code>stack</code> vs <code>queue</code>	LIFO vs FIFO
<code>lower_bound</code> vs <code>upper_bound</code>	$\geq x$ vs $> x$

Must-Remember Facts

- `sort(v.begin(), v.end())` works on vectors, strings, arrays, and random-access ranges.
- `lower_bound(v.begin(), v.end(), x) - v.begin()` gives index in vector.
- `set.lower_bound(x)` gives iterator to first element $\geq x$.
- `priority_queue<pair<int,int>>` compares pairs lexicographically by default.
- Use `greater<int>` for min heap.

Interview Traps

- Do not use `lower_bound` on unsorted vector.
- Do not dereference `end()`.
- Do not rely on order in `unordered_map`.
- Do not assume `unordered_map` worst-case is $O(1)$.
- Do not modify a container carelessly while iterating.

13. Practice Tasks

Beginner Tasks

1. Use `vector` to store marks of students and print the maximum.
2. Use `string` to count vowels in a word.
3. Store student roll number and name using `pair`.
4. Store employee ID, name, and salary using `tuple`.
5. Use `stack` to reverse a string.

Intermediate Tasks

1. Check balanced parentheses using `stack`.
2. Print level order traversal of a binary tree using `queue`.
3. Find first non-repeating character using `queue` and `unordered_map`.
4. Find top K largest elements using `priority_queue`.
5. Count frequency of elements using `unordered_map`.
6. Remove duplicates and print sorted elements using `set`.
7. Find the next greater element using `stack`.
8. Find sliding window maximum using `deque`.

Advanced Tasks

1. Implement Dijkstra's algorithm using `priority_queue`.
2. Find longest increasing subsequence length using `lower_bound`.
3. Merge K sorted arrays using `priority_queue`.
4. Solve two-sum using `unordered_map`.
5. Find number of subarrays with sum K using `unordered_map`.
6. Schedule meetings using sorting and `priority_queue`.
7. Use `map` to process range events with sweep line.
8. Use `set.lower_bound` to find closest greater or equal element.

Small C++ Practice Program

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> v = {5, 1, 3, 3, 2};

    sort(v.begin(), v.end());

    set<int> uniqueSorted(v.begin(), v.end());

    unordered_map<int, int> freq;
    for (int x : v) freq[x]++;

    auto it = lower_bound(v.begin(), v.end(), 3);

    cout << "Sorted vector: ";
    for (int x : v) cout << x << " ";
    cout << "\n";

    cout << "Unique sorted values: ";
    for (int x : uniqueSorted) cout << x << " ";
    cout << "\n";

    cout << "Frequency of 3: " << freq[3] << "\n";

    if (it != v.end()) {
        cout << "First value >= 3 is at index " << (it - v.begin()) << "\n";
    }
}
```

14. Final Cheat Sheet

Core Definition

STL in C++ is a template-based library that provides common containers, algorithms, and iterators for efficient programming.

Why It Matters

It helps solve DSA and placement problems faster by giving optimized implementations of common data structures.

Most Asked Questions

Question	Short Answer
Vector vs array?	Vector is dynamic, array is fixed-size
Set vs unordered_set?	Sorted $O(\log n)$ vs unordered average $O(1)$
Map vs unordered_map?	Sorted keys vs hash-based lookup
Stack vs queue?	LIFO vs FIFO
Default priority queue?	Max heap
Min heap syntax?	<code>priority_queue<int, vector<int>, greater<int>></code>
<code>lower_bound</code> meaning?	First element $\geq x$
<code>upper_bound</code> meaning?	First element $> x$
What is iterator?	Generalized pointer into a container
What does <code>mp[key]</code> do?	Accesses value, inserts default if missing

Common Comparisons

Need	Use
Dynamic array	<code>vector</code>
Text	<code>string</code>
Two values	<code>pair</code>
Multiple values	<code>tuple</code>
LIFO	<code>stack</code>
FIFO	<code>queue</code>
Both-end operations	<code>deque</code>
Highest or lowest priority	<code>priority_queue</code>
Sorted unique values	<code>set</code>
Sorted key-value pairs	<code>map</code>
Fast unique lookup	<code>unordered_set</code>
Fast key-value lookup	<code>unordered_map</code>
Binary search position	<code>lower_bound</code>

One-Line Interview Answer

STL is C++'s ready-made toolkit of generic containers, algorithms, and iterators; in interviews, I choose among `vector`, `set`, `map`, `unordered_map`, `stack`, `queue`, `priority_queue`, and related tools based on ordering, duplicates, access pattern, and time complexity.